深度學習硬體架構

Objective:

- Problem:

本研究的問題是，如何使用一個類神經網路 (Neural Network) 來學習一個非線性的三維函數。

- Purpose:

  - 實作一個具有單一隱藏層的倒傳遞 (Back-propagation, BP) 演算法。
  - 訓練此網路來模擬 (逼近) 目標函數$f(x_1, x_2) = x_1^2 + x_2^2$，其中輸入範圍為$x_1, x_2 \in [-5, 5]$。
  - 探討隱藏層單元數量 (p)（例如 p=5, 10, 20）對於網路學習成效、收斂速度與最終逼近能力的影響。

Procedure:

- Method:

本實驗採用具備動量 (Momentum) 項的批次 (Batch) 倒傳遞演算法。在每次迭代 (iteration) 中，網路會計算所有 300 筆訓練資料的總誤差梯度，並進行一次權重更新。動量（alpha = 0.9）的加入是為了加速收斂並協助跳脫局部最小值。

- Network Structure:

- 輸入層 (Input Layer)： 2 個神經元（x_1和x_2）。
- 隱藏層 (Hidden Layer)： 1 層，具備 p 個神經元（本實驗 p 設定為 5, 10, 20）。
- 輸出層 (Output Layer)： 1 個神經元（代表網路預測的$\hat{f}(x_1, x_2)$）。
- 激活函數 (Activation Function)： 隱藏層與輸出層均使用 Logistic (Sigmoid) 函數，$\varphi(v) = 1/(1 + \mathrm{e}^{-v})$。

- Program Flow Chart:

```python
# --- 1. 設定共同參數 ---
learning_rate = 1
iterations = 30000  # 訓練次數
alpha = 0.9         # 動量參數
bias = 1            # 偏差
input_size = 2      # 輸入層大小
output_size = 1     # 輸出層大小

train_pattern_num = 300  # 訓練資料數目
test_pattern_num = 100   # 測試資料數目
```

初始化： 設定學習率 ($\eta = 1$)、動量 ($\alpha = 0.9$)、迭代次數 (30,000)。

生成 300 筆訓練資料 (train) 和 100 筆測試資料 (test)。

```python
# --- 3. 生成並準備資料 (移到迴圈外) ---
# 確保每次實驗都用同一組資料
np.random.seed(42) # 固定隨機種子以便重現結果
x = np.random.uniform(-5, 5, (total_pattern_num, input_size))
d = np.zeros((total_pattern_num, output_size))
d[:, 0] = ideal_output(x[:, 0], x[:, 1])

x_train = x[:train_pattern_num, :]
d_train = d[:train_pattern_num]
x_test = x[train_pattern_num:, :]
d_test = d[train_pattern_num:]

# 預先縮放訓練和測試的目標值
d_train_scaled = (((d_train - 0) * (0.8 - 0.2)) / (50 - 0)) + 0.2
d_test_scaled = (((d_test - 0) * (0.8 - 0.2)) / (50 - 0)) + 0.2
```

資料縮放 (Scaling)： 由於 Logistic 函數輸出為 [0, 1]，為使網路收斂，將理想輸出d縮放至 [0.2, 0.8] 區間，作為網路的訓練目標d_{scaled}。

```python
# --- 4. 建立實驗迴圈 ---

hidden_units_list = [5, 10, 20]  # 要測試的隱藏單元數量
results = []                     # 儲存結果以便製作表格

for hidden_size in hidden_units_list:

    print(f"\n--- 正在開始實驗: 隱藏單元 (p) = {hidden_size} ---")

    # --- 5. 初始化權重 (每次實驗都要重新初始化) ---
    W_hidden = np.random.uniform(-0.3, 0.3, (hidden_size, input_size))
    W_output = np.random.uniform(-0.3, 0.3, (output_size, hidden_size))

    dW_hidden_last = np.zeros_like(W_hidden)
    dW_output_last = np.zeros_like(W_output)

    # --- 6. 訓練過程 ---
    MSE_in_iterations = []
    for itr in range(iterations):
        # feedforward propagation
        V_hidden = np.dot(x_train, W_hidden.T) + bias
        y_hidden = logistic_function(V_hidden)
        V_output = np.dot(y_hidden, W_output.T) + bias
        y_output = logistic_function(V_output)

        # 計算誤差 (使用縮放後的資料)
        error = d_train_scaled - y_output
        MSE_in_iterations.append(MSE(d_train_scaled, y_output))

        # backpropagation
        delta_output = error * Phi(V_output)
        dW_output = (learning_rate * np.dot(delta_output.T, y_hidden)) / train_pattern_num + alpha * dW_output_last
        delta_hidden = np.dot(delta_output, W_output) * Phi(V_hidden)
        dW_hidden = (learning_rate * np.dot(delta_hidden.T, x_train)) / train_pattern_num + alpha * dW_hidden_last
```

```python
        # 更新權重
        W_hidden += dW_hidden
        W_output += dW_output

        dW_hidden_last = dW_hidden
        dW_output_last = dW_output

    # 訓練完的平均誤差
    final_train_mse = MSE_in_iterations[-1]
    print(f'訓練完的 Eav = {final_train_mse}')

    # --- 7. 測試過程 ---
    V_hidden_test = np.dot(x_test, W_hidden.T) + bias
    y_hidden_test = logistic_function(V_hidden_test)
    V_output_test = np.dot(y_hidden_test, W_output.T) + bias
    y_test_output = logistic_function(V_output_test)

    # 測試階段的平均誤差 (使用縮放後的資料)
    test_error = d_test_scaled - y_test_output
    test_MSE = (0.5 * (test_error ** 2)).mean()
    print(f'測試時的 Eav = {test_MSE}')

    # 儲存結果
    results.append({
        "hidden_units": hidden_size,
        "train_mse": final_train_mse,
        "test_mse": test_MSE
    })
```

結束實驗迴圈。

```python
# --- 8. 繪圖 (每次實驗都畫一次) ---

# 建立 meshgrid 以進行輸出計算
x1_range = np.linspace(-5, 5, 50)
x2_range = np.linspace(-5, 5, 50)
x1_mesh, x2_mesh = np.meshgrid(x1_range, x2_range)
x_mesh_flattened = np.c_[x1_mesh.ravel(), x2_mesh.ravel()]

# 前向傳播計算 meshgrid 上的輸出
V_hidden_mesh = np.dot(x_mesh_flattened, W_hidden.T) + bias
y_hidden_mesh = logistic_function(V_hidden_mesh)
V_output_mesh = np.dot(y_hidden_mesh, W_output.T) + bias
y_output_mesh = logistic_function(V_output_mesh)

# 重塑 meshgrid 輸出為 2D，並縮放回原始範圍
y_output_mesh_reshaped = (((y_output_mesh - 0.2) * 50) / 0.6).reshape(x1_mesh.shape)

# 計算理想輸出的 meshgrid surface
ideal_output_mesh = ideal_output(x1_mesh, x2_mesh)

# 繪圖
fig = plt.figure(figsize=(15, 10))
# 加上一個總標題，說明這是哪個 p 的結果
fig.suptitle(f'Test Result: {hidden_size} hidden units (p={hidden_size})', fontsize=16)

ax1 = fig.add_subplot(221)
ax1.plot(range(1, iterations + 1), MSE_in_iterations)
ax1.set_title("MSE over Iterations")
ax1.set_xlabel("Iterations")
ax1.set_ylabel("MSE")

ax2 = fig.add_subplot(222, projection='3d')
ax2.plot_surface(x1_mesh, x2_mesh, ideal_output_mesh, color='lightgray', alpha=0.5)
ax2.set_title("Ideal Output Surface")
ax2.set_xlabel("X1")
ax2.set_ylabel("X2")
ax2.set_zlabel("Output")
```

印出比較 p=5, 10, 20 的總結表格，並繪製 3D 曲面圖

Calculation (關鍵計算 - 權重更新)：

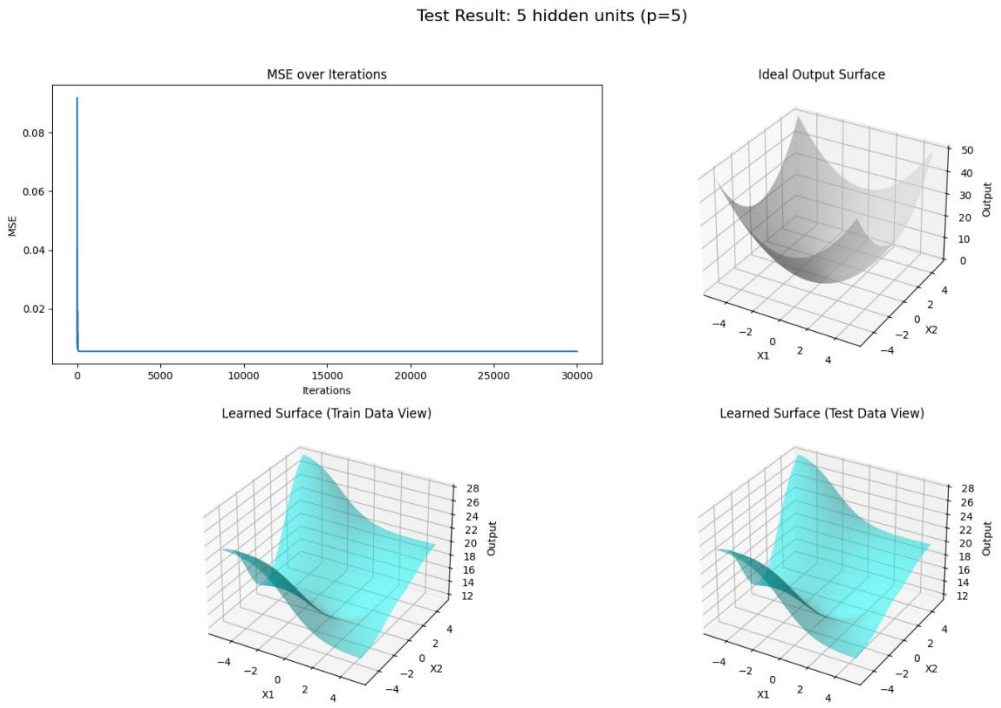
本程式使用「批次更新」，權重變化量(dW)是根據 300 筆資料的平均梯度計算而來：

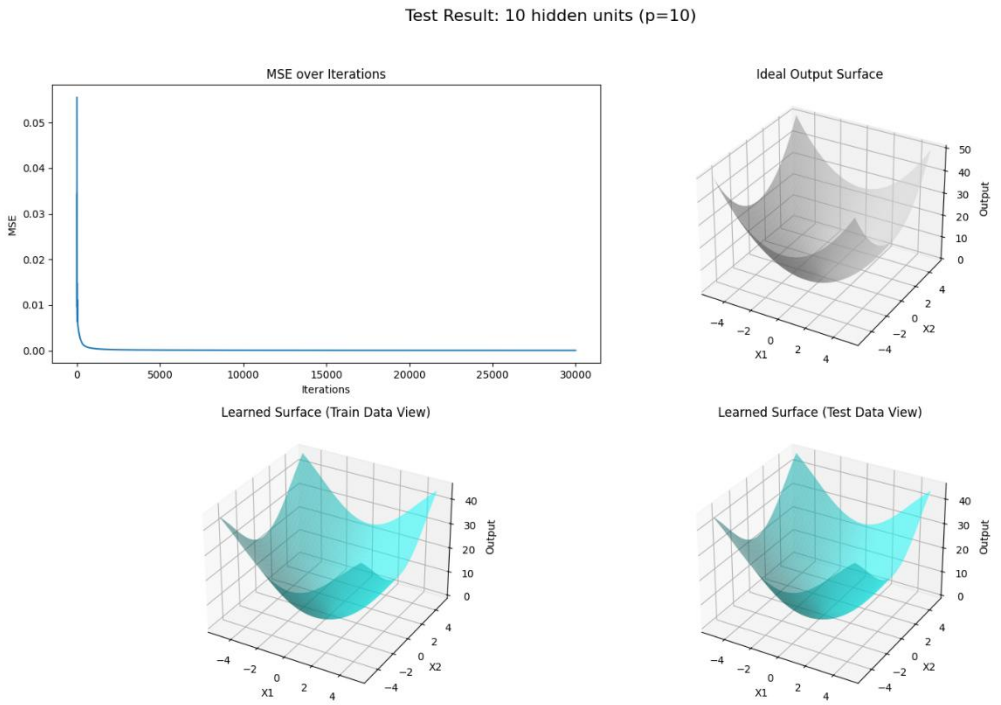
$$\Delta W_{kj}(t) = \frac{\eta}{N}\sum_{n=1}^{N}\delta_k^{(n)}y_j^{(n)} + \alpha\Delta W_{kj}(t-1)$$

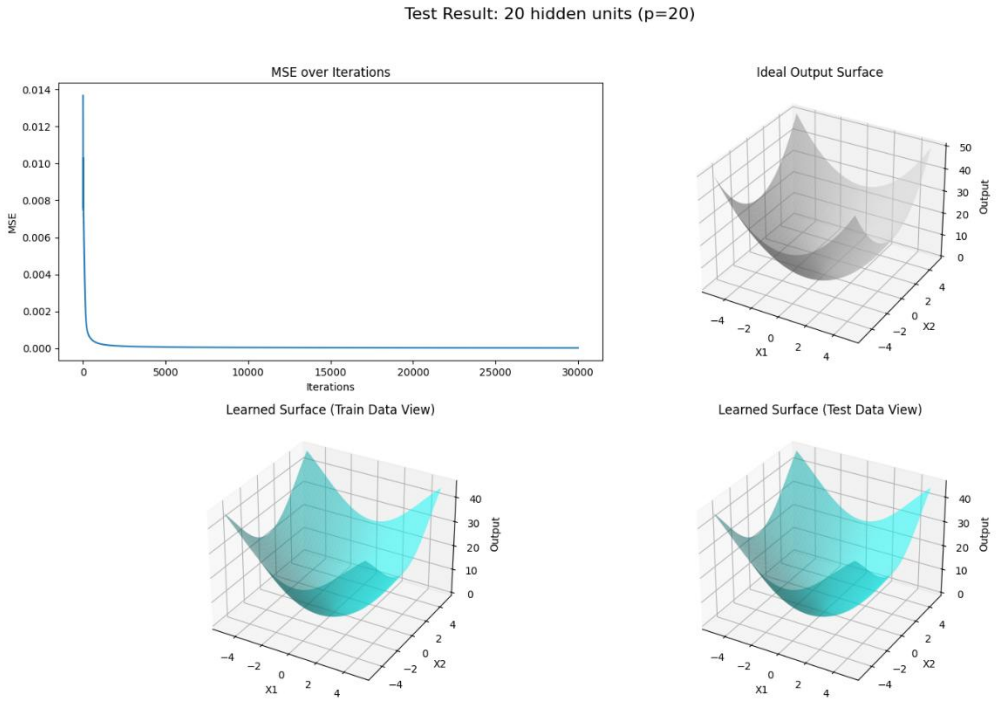
$$\Delta W_{ji}(t) = \frac{\eta}{N}\sum_{n=1}^{N}\delta_j^{(n)}x_i^{(n)} + \alpha\Delta W_{ji}(t-1)$$

其中$N = 300$(train_pattern_num)。

Simulation results:







左上 (MSE over Iterations)：

功能： 顯示學習的「過程」。它記錄了訓練過程中，每一次迭代的均方誤差 (MSE)。

- 在 p=10 和 p=20 的情況下，曲線在初期呈「懸崖式」下降，這代表網路（由於隨機初始權重）一開始的誤差非常高，但很快就找到了正確的下降方向。
- 在 p=5 的情況下，雖然曲線也下降了，但最終收斂的 MSE 值明顯高於 p=10 和 p=20，代表它卡住 (stuck) 了，無法再進一步最佳化。

左下 (Learned Surface (Train Data View))：

功能： 這是「學習成果」。它顯示了訓練完成後的神經網路所代表的函數 $f_{NN}(x_1, x_2)$。

- 這是最重要的比較圖。在 p=5 時，這個曲面是錯誤的，可能只是一個扭曲的平面，完全沒有學到拋物面的特徵。
- 在 p=10 和 p=20 時，這個曲面與「Ideal Output Surface」幾乎完美重疊，證明網路成功抓到了$x_1^2 + x_2^2$的非線性特徵。

```
>>> %Run simulation1.py

  --- 正在開始實驗: 隱藏單元 (p) = 5 ---
  訓練完的 Eav = 0.005415080505923448
  測試時的 Eav = 0.003868170166526015

  --- 正在開始實驗: 隱藏單元 (p) = 10 ---
  訓練完的 Eav = 4.0214516835295445e-05
  測試時的 Eav = 3.716210967160002e-05

  --- 正在開始實驗: 隱藏單元 (p) = 20 ---
  訓練完的 Eav = 2.8435735872032913e-05
  測試時的 Eav = 2.6198459682810777e-05


  ================================================================
                    實驗結果總結 (for Table)
  ================================================================
  | 隱藏單元數量 (p)      | 訓練完的 MSE (Eav)       | 測試時的 MSE (Eav)       |
  |-----------------|----------------------|----------------------|
  | 5               | 5.4150805059e-03     | 3.8681701665e-03     |
  | 10              | 4.0214516835e-05     | 3.7162109672e-05     |
  | 20              | 2.8435735872e-05     | 2.6198459683e-05     |
  ================================================================
```

Conclusion:

➢ Analysis and Comparison:

隱藏層單元 (p) 的影響：

- p = 5 (Underfitting / 學習不足)： 如 3D 圖所示，網路完全無法擬合 (fit) $x_1^2 + x_2^2$的曲面。表格中的 MSE(10^{-3})也證實了這一點，相較於 p=10，誤差大了 100 倍以上。這表明 5 個隱藏單元不足以表達此函數所需的複雜度。
- p = 10 (Sufficient / 學習充足)： 網路成功收斂，3D 學習曲面與理想曲面完美貼合。訓練與測試的 MSE 都達到了(10^{-5})的極低水平。這表明 10 個隱藏單元足以模擬此函數。
- p = 20 (Diminishing Returns / 效益遞減)： 網路同樣完美收斂，MSE 甚至比 p=10 略低 (2.6e-05 vs 3.7e-05)。然而，將網路複雜度增加一倍（從 10 到 20），錯誤率的改善卻很有限。

總結：

本實驗成功使用倒傳遞演算法，讓神經網路學習了$f(x_1, x_2) = x_1^2 + x_2^2$函數。實驗證明，隱藏層的規模（神經元數量 p）是影響學習成敗的關鍵因素。當p過少 (p=5)，網路會學習失敗；當p足夠 (p=10)，網路可以完美模擬目標；而當p過多 (p=20)，雖然效果略好，但也帶來了額外的計算成本，呈現效益遞減。